



Lập trình CSharp (C#)

Phương thức

- Khai báo phương thức (hàm)
- Truyền tham số dạng **in (ø)**
- Truyền tham số dạng **out**
- Truyền tham số dạng **ref**

Khái báo phương thức

```
[modifiers] return_type MethodName([parameters])
```

```
{  
    // Thân phương thức  
}
```

Ví dụ:

```
public static void Xuat(StrHocSinh hs)  
{  
    Console.WriteLine("Ma so: {0}. Ho ten: {1}", hs.MaSo, hs.HoTen);  
    //Cau lenh xuất học sinh  
}
```

Phương thức dạng "in"

- Thân phương thức chỉ tham khảo giá trị của tham số không thay đổi giá trị của tham số

- Ví dụ:

```
public static void Xuat(StrHocSinh hs)
{
    Console.WriteLine("Ma so: {0}. Ho ten: {1}", hs.MaSo, hs.HoTen);
    //Cau lenh xuat hoc sinh
}
```

- Gọi hàm trong hàm Main:

```
Xuat(hs);
```

Phương thức dạng "out"

- Thân phương thức cấp phát (khởi tạo) giá trị của tham số trước khi sử dụng. Ra khỏi hàm giá trị của tham số thay đổi.

- Ví dụ:

```
public static void Nhap(out StrHocSinh hs)
{
    hs = new StrHocSinh();
    //Cau lenh nhap hoc sinh
}
```

- Gọi trong hàm Main:

```
Nhap(out hs);
```

Phương thức dạng "ref"

- Ra khỏi hàm giá trị của tham số sẽ thay đổi

- Ví dụ:

```
public static void TinhDiemTrungBinh(ref StrHocSinh hs)
{
    hs.DTB = (hs.Toan + hs.Van) / 2;
}
```

- Gọi trong hàm Main:

```
TinhDiemTrungBinh(ref hs);
```

Struct

- Struct là kiểu Value Type không phải là Reference Type => có thể không cần sử dụng từ khóa new.
- Trong Struct có thể định nghĩa các phương thức (giống Class).
- Trong Struct, trình biên dịch luôn luôn cung cấp một constructor không tham số mặc định, và không cho phép thay thế.
- Struct không hỗ trợ thừa kế.

Struct

```
struct StrHocSinh
```

```
{
```

```
    public int MaSo;
```

```
    public string HoTen;
```

```
    public double Toan;
```

```
    public double Van;
```

```
    public double DTB;
```

```
    public StrHocSinh(int ms, string ht, double t, double v)
```

```
    {
```

```
        MaSo = ms;
```

```
        HoTen = ht;
```

```
        Toan = t;
```

```
        Van = v;
```

```
        DTB = (t+ v)/2;
```

```
    }
```

```
}
```

Mảng

- Mảng 1 chiều
- Mảng 2 chiều
- Mảng nhiều chiều
- Mảng Jagged Array

Mảng 1 chiều

- Cú pháp:

`type[ ] array-name;`

- Ví dụ:

`int[] myIntegers; // mảng kiểu số nguyên`

`integers = new int[32];`

`integers[0] = 35; // phần tử đầu tiên có giá trị 35`

`integers[31] = 432; // phần tử 32 có giá trị 432`

`string[] myArray = {"first element", "second element",
"third element"};`

Làm việc với mảng 1 chiều

- Lấy kích thước mảng:

```
int arrayLength = myIntegers.Length;
```

- Sắp xếp mảng số nguyên:

```
Array.Sort(myIntegers);
```

- Đảo ngược mảng:

```
Array.Reverse(myArray);
```

- Duyệt mảng:

Mảng 2 chiều

- Cú pháp:

`type[,] array-name;`

- Ví dụ:

```
int[,] myRectArray = new int[2,3];
```

```
int[,] myRectArray = new int[,]{ {1,2},{3,4},{5,6},{7,8}};  
//mảng 4 hàng 2 cột
```

```
string[,] beatleName = { {"Lennon","John"},  
    {"McCartney","Paul"}, {"Harrison","George"},  
    {"Starkey","Richard"} };
```

Làm việc với mảng 2 chiều

- Duyệt mảng:

```
double [, ] matrix = new double[10, 10];  
for (int i = 0; i < 10; i++)  
{  
    for (int j=0; j < 10; j++)  
        matrix[i, j] = 4;  
}
```

Mảng nhiều chiều

- Ví dụ:

```
string[,] my3DArray;
```

Mảng jagged

- Một loại thứ 2 của mảng nhiều chiều trong C# là Jagged array.
- Ví dụ:

```
int[][] a = new int[3][];
```

```
a[0] = new int[4];
```

```
a[1] = new int[3];
```

```
a[2] = new int[1];
```

Làm việc với Jagged Array

- Khởi tạo ma trận $n \times m$ Jagged Array:

```
int[][] a = new int[n][];  
for(int i = 0; i < n; i++)  
{  
    a[i] = new int[m];  
    for (int j = 0; j < m; j++)  
    {  
        a[i][j] = i*n + j;  
    }  
}
```

Bài tập

- Nhóm bài tập chỉ có khối hàm Main:
 - Các bài tập về mảng
- Nhóm bài tập có: khối hàm Main, hàm tự định nghĩa, kiểu dữ liệu định nghĩa (struct)
 - Các bài tập lập trình hướng đơn thể
 - Các bài tập về mảng